

CSA1718 AI
LAB PROGRAMS

SANJAI S(192511018)

EXPERIMENT 21

BIRD CAN FLY OR NOT

AIM

To write a Prolog program to determine whether a particular bird can fly or not and execute the required queries.

ALGORITHM

1. Define facts for birds.
2. Define facts for birds that can fly.
3. Define facts for birds that cannot fly.
4. Create a rule to determine whether a bird can fly.
5. Query the required bird.
6. Display whether the bird can fly or not.

PSEUDOCODE

START

Define bird facts

Define flying bird facts

Define non-flying bird facts

IF bird belongs to flying birds

THEN bird can fly

ELSE IF bird belongs to non-flying birds

THEN bird cannot fly

ELSE

Result is unknown

Perform required queries

STOP

The screenshot shows the SWISH Prolog IDE interface. On the left, the 'Program' tab contains the following code:

```
1 bird(sparrow).
2 bird(penguin).
3 bird(parrot).
4 can_fly(sparrow).
5 can_fly(parrot).
6 flies(X) :-
7     bird(X),
8     can_fly(X).
```

On the right, the 'Results' pane shows the execution of the query `flies(penguin).`. The output is as follows:

```
Current predicate: flies/1
Use :- discontiguous can_fly/1. to suppress this message

true
?- flies(penguin).

true
?- flies(penguin).

true
?- flies(penguin).

true
?- flies(penguin).

false
?- flies(penguin).

false
?- flies(penguin).
```

The results pane also includes a search bar, a 'Search' button, and a 'Run' button at the bottom right.

RESULT

The Prolog program successfully determines whether a given bird can fly or not using facts and rules.

EXPERIMENT 22

FAMILY TREE

AIM

To write a Prolog program to implement a family tree and determine relationships between family members.

ALGORITHM

1. Define male and female facts.
2. Define parent relationships.
3. Create rules for father, mother, brother, sister, grandfather, grandmother, etc.
4. Query the required relationship.
5. Display the result.

PSEUDOCODE

START

Define male and female persons

Define parent relationships

IF X is parent of Y AND X is male

THEN X is father of Y

IF X is parent of Y AND X is female

THEN X is mother of Y

IF X and Y have the same parent

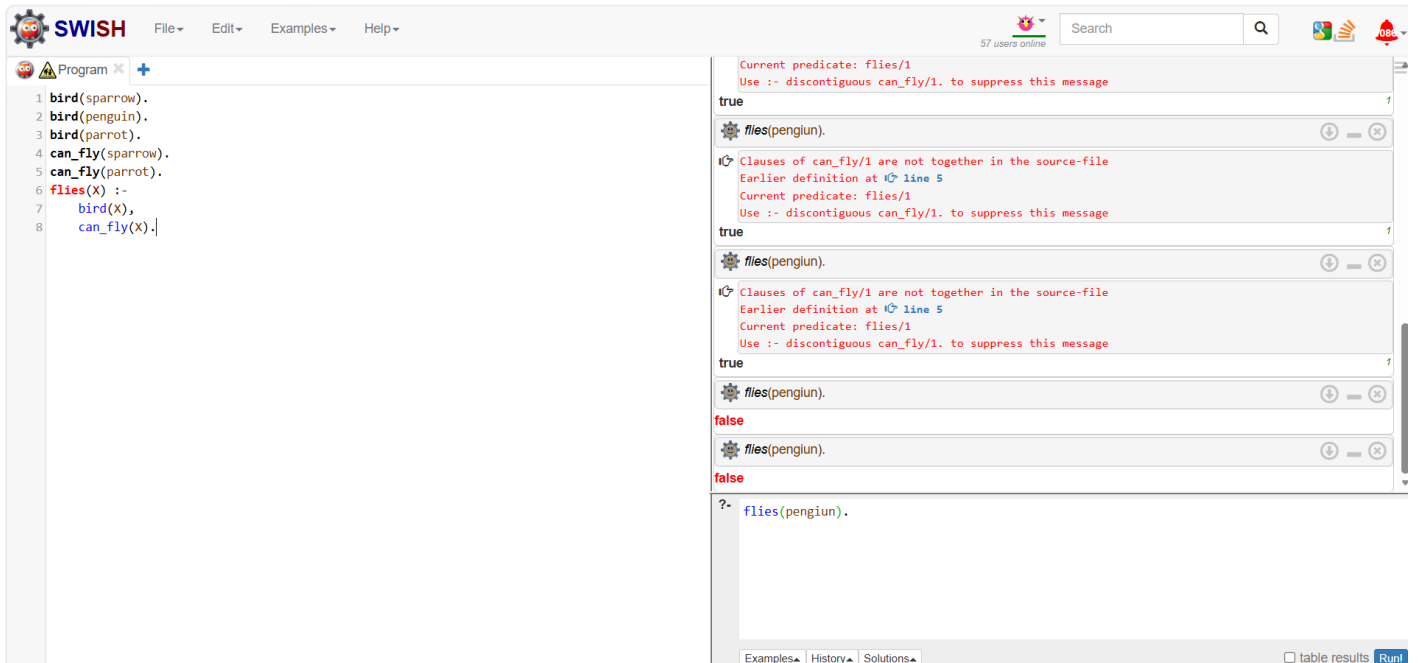
THEN X and Y are siblings

IF X is parent of Y AND Y is parent of Z

THEN X is grandparent of Z

Perform relationship queries

STOP



The screenshot shows the SWISH Prolog environment. On the left, a code editor displays a Prolog program with the following lines:

```
1 bird(sparrow).
2 bird(penguin).
3 bird(parrot).
4 can_fly(sparrow).
5 can_fly(parrot).
6 flies(X) :-
7     bird(X),
8     can_fly(X).
```

On the right, the execution results are shown. The first two queries, `flies(sparrow).` and `flies(parrot).`, both return `true`. The third query, `flies(penguin).`, returns `false`. The results are displayed in a table format with columns for the query and the result.

Query	Result
<code>flies(sparrow).</code>	<code>true</code>
<code>flies(parrot).</code>	<code>true</code>
<code>flies(penguin).</code>	<code>false</code>

RESULT

The Prolog program successfully represents a family tree and identifies relationships between family members.

EXPERIMENT 23

DIETING SYSTEM BASED ON DISEASE

AIM

To write a Prolog program that suggests a suitable diet based on the disease of a person.

ALGORITHM

1. Define diseases.
2. Define recommended food for each disease.
3. Define foods that should be avoided.
4. Create rules connecting disease with diet.
5. Accept/query the disease.
6. Display the recommended diet.
- 7.

PSEUDOCODE

START

Define disease facts

Define recommended food for each disease

Define food to avoid for each disease

INPUT disease

IF disease has recommended food

THEN display recommended food

AND display food to avoid

ELSE

Display "No diet information available"

STOP

The screenshot shows the SWISH Prolog IDE interface. On the left, the 'Program' tab contains the following code:

```
1 bird(sparrow).
2 bird(penguin).
3 bird(parrot).
4 can_fly(sparrow).
5 can_fly(parrot).
6 flies(X) :-
7     bird(X),
8     can_fly(X).
```

On the right, the 'Solutions' tab displays the execution results for the query `flies(penguin).`. The results are as follows:

Solution
1 true
2 false
3 false

The bottom of the interface shows tabs for 'Examples', 'History', and 'Solutions', along with a 'Run!' button and a checkbox for 'table results'.

RESULT

The Prolog program successfully suggests a suitable diet according to the given disease.

EXPERIMENT 24

MONKEY BANANA PROBLEM

AIM

To write a Prolog program to solve the Monkey Banana Problem using logical rules and state transitions.

ALGORITHM

1. Represent the initial state of the monkey, box, banana and floor.
2. Define actions such as moving, pushing the box and climbing.
3. Define the condition for obtaining the banana.
4. Apply valid actions recursively.
5. Stop when the monkey obtains the banana.

Pseudocode

START

Define initial state:

Monkey on floor

Box at some position

Banana at another position

IF monkey can move

THEN move monkey

IF monkey is near box

THEN push box

IF box is below banana

THEN climb onto box

IF monkey is on box AND box is below banana

THEN monkey gets banana

Display sequence of actions

STOP

Result

The Prolog program successfully finds a sequence of actions through which the monkey obtains the banana.

EXPERIMENT 25 — Fruit and Its Color Using Backtracking

Aim

To write a Prolog program to identify the color of a fruit using backtracking.

Algorithm

- 1. Define fruit-color facts.**
- 2. Query a fruit.**
- 3. Prolog searches for a matching fact.**
- 4. If multiple answers exist, use backtracking to find additional answers.**
- 5. Display the fruit and its color.**

Pseudocode

START

Define fruit and color relationships

INPUT fruit

Search for matching fruit

IF matching fruit is found

THEN display its color

ELSE

Backtrack and search for another matching fact

IF no more matches

THEN display "No match found"

STOP

Result

The Prolog program successfully identifies fruit colors and demonstrates the backtracking mechanism.

EXPERIMENT 26 — Best First Search Algorithm

Aim

To write a Prolog program to implement the Best First Search algorithm.

Algorithm

- 1. Define the starting node and goal node.**
- 2. Define the heuristic value of each node.**
- 3. Insert the starting node into the open list.**
- 4. Select the node with the lowest heuristic value.**
- 5. Check whether it is the goal.**
- 6. If not, generate its neighboring nodes.**
- 7. Add the neighbors to the open list.**
- 8. Repeat until the goal is reached.**

Pseudocode

START

OPEN = {start node}

CLOSED = empty

WHILE OPEN is not empty

Select node N having the lowest heuristic value

Remove N from OPEN

Add N to CLOSED

IF N is the goal

THEN display path

STOP

Generate successors of N

Add unvisited successors to OPEN

END WHILE

Display "Goal not found"

STOP

Result

The Prolog program successfully implements Best First Search and finds a path toward the goal using heuristic values.

Aim

To write a Prolog program for medical diagnosis based on symptoms.

Algorithm

- 1. Define symptoms associated with diseases.**
- 2. Accept symptoms from the user.**
- 3. Compare the symptoms with disease rules.**
- 4. Identify the disease whose conditions are satisfied.**
- 5. Display the possible diagnosis.**

Pseudocode

START

Define disease rules

Define symptoms for each disease

INPUT symptoms

FOR each disease

Check whether required symptoms are present

IF all required symptoms are present

THEN identify possible disease

END FOR

Display diagnosis

STOP

Result

The Prolog program successfully identifies a possible disease based on the symptoms provided.

Note: This is only a Prolog/AI demonstration, not a real medical diagnosis system.

EXPERIMENT 28 — Forward Chaining

Aim

To write a Prolog program to demonstrate forward chaining and execute the required queries.

Algorithm

1. Define initial facts.
2. Define rules.
3. Start with the available facts.
4. Check which rules can be applied.
5. Add newly derived facts to the knowledge base.
6. Continue until no new facts can be generated.
7. Check the required query.

Pseudocode

START

Define initial facts

Define rules

REPEAT

FOR each rule

IF all conditions of rule are true

THEN derive the conclusion

Add new conclusion to knowledge base

END FOR

UNTIL no new facts are generated

Check query

IF query is a derived fact

THEN display "True"

ELSE

display "False"

STOP

Result

The Prolog program successfully demonstrates forward chaining by deriving new facts from existing facts and rules.

EXPERIMENT 29 — Backward Chaining

Aim

To write a Prolog program to demonstrate backward chaining and execute the required queries.

Algorithm

- 1. Define facts and rules.**
- 2. Select the required goal.**

3. Search for a rule that can prove the goal.
4. Treat the rule's conditions as sub-goals.
5. Recursively prove each sub-goal.
6. If all sub-goals are satisfied, the goal is true.
7. Otherwise, backtrack and try another rule.

Pseudocode

START

Define facts

Define rules

INPUT goal

IF goal is a known fact

THEN goal is TRUE

ELSE

Find a rule whose conclusion matches the goal

FOR each condition in the rule

Try to prove the condition recursively

IF all conditions are true

THEN goal is TRUE

ELSE

Backtrack and try another rule

IF goal cannot be proved

THEN goal is FALSE

STOP

Result

The Prolog program successfully demonstrates backward chaining by starting from the goal and working backward toward the required facts.

EXPERIMENT 30 — Web Blog Using WordPress

Aim

To create a web blog using WordPress and demonstrate HTML elements such as Anchor Tag, Title Tag, headings, paragraphs and links.

Algorithm

- 1. Open WordPress.**
- 2. Create a new blog/site.**
- 3. Add a suitable title.**
- 4. Create a new post/page.**
- 5. Add headings and paragraphs.**
- 6. Insert an anchor/link.**
- 7. Add required content.**
- 8. Preview the page.**
- 9. Publish the blog.**

Pseudocode

START

Open WordPress

Create new blog

Set website title

Create new post

Add heading

Add paragraph

Add image if required

Create anchor link

Link Text

Add other required HTML elements

Preview webpage

IF webpage is correct

THEN publish webpage

ELSE

Edit webpage

STOP

Result

A web blog was successfully created using WordPress with title, heading, paragraph and anchor/link elements.

Aim

To write a Prolog program to implement pattern matching.

Algorithm

- 1. Define the input pattern.**
- 2. Define the data or target string/list.**
- 3. Compare the pattern with the target.**
- 4. Match corresponding elements.**
- 5. If all elements match, return success.**
- 6. Otherwise, backtrack or return failure.**

Pseudocode

START

INPUT pattern

INPUT target

Compare pattern with target

IF pattern is empty AND target is empty

THEN MATCH SUCCESS

ELSE IF first elements match

THEN recursively match remaining elements

ELSE

MATCH FAILURE

Display result

STOP

Result

The Prolog program successfully performs pattern matching and determines whether the given pattern matches the target.

EXPERIMENT 32 — Number of Vowels

Aim

To write a Prolog program to find the number of vowels in a given word or string.

Algorithm

- 1. Read the input string/list.**
- 2. Check each character.**
- 3. If the character is a vowel, increment the count.**
- 4. Continue until all characters are processed.**
- 5. Display the total number of vowels.**

Pseudocode

START

INPUT word

COUNT = 0

FOR each character in word

IF character is a, e, i, o or u

THEN COUNT = COUNT + 1

END FOR

Display COUNT

STOP

Result

The Prolog program successfully counts and displays the number of vowels present in the given word/string.